Avaliação de Desempenho de Microsserviços com Locust

Trabalho 06 — Sistemas Distribuídos

Gabriel Alves de Freitas

Outubro de 2026

1 Introdução e Objetivo

Em sistemas distribuídos contemporâneos, a arquitetura orientada a microsserviços tornou-se o paradigma dominante para a construção de aplicações escaláveis, desacopladas e resilientes. Contudo, a transição de monólitos para grafos distribuídos de serviços introduz novos desafios operacionais e arquiteturais, tais como a latência de rede entre nós, a gestão de pools de conexões e o enfileiramento em pontos centrais de entrada (*API Gateways*).

O objetivo deste trabalho prático é realizar uma rigorosa avaliação de desempenho da aplicação **Spring PetClinic Microservices**[1] utilizando a ferramenta de testes de carga distribuída **Locust**. A investigação foca em analisar:

i. O impacto da escalabilidade da carga pelo aumento progressivo de usuários simultâneos concorrentes (30, 80 e 150 usuários);

ii. A influência da composição semântica do tráfego nas métricas de qualidade de serviço (QoS), comparando perfis dominados por leitura contra perfis dominados por persistência de escrita;

iii. A distribuição do consumo de recursos computacionais (notadamente CPU) entre os microsserviços constituintes da malha;

iv. O comportamento da arquitetura sob estresse limite, incluindo o surgimento de falhas em cascata e o disparo de proteções de circuito.

Toda a infraestrutura de microsserviços foi orquestrada utilizando **Docker Compose**, garantindo isolamento de processos e reprodutibilidade experimental.

2 Arquitetura da Aplicação e Mapeamento de Endpoints

A aplicação avaliada é a versão de microsserviços da referência arquitetural Spring PetClinic, construída com Spring Boot 3 e Spring Cloud. A topologia é composta pelos seguintes elementos principais:

[1] https://github.com/spring-petclinic/spring-petclinic-microservices

- **Config Server** (porta 8888): Servidor centralizado de configurações externas.
- **Discovery Server** (Eureka, porta 8761): Registro e localização dinâmica de instâncias de microsserviços.
- **API Gateway** (Spring Cloud Gateway, porta 8080): Ponto de contato público que atua como proxy reverso reativo, roteador e balanceador de carga integrado ao Eureka.
- **Customers Service** (porta 8081): Microsserviço de domínio responsável pela gestão de donos (*owners*) e animais (*pets*), conectado a um banco de dados relacional dedicado.
- **Vets Service** (porta 8083): Microsserviço de domínio responsável pela gestão de veterinários e suas especialidades médicas, com banco de dados independente.
- **Visits Service** (porta 8082): Microsserviço de domínio de agendamentos e registros clínicos de consultas.

2.1 Verificação dos Endpoints na Versão Utilizada

Conforme exigido na especificação do projeto, todos os endpoints de teste foram rigorosamente inspecionados no código-fonte e na configuração do `api-gateway` (`application.yml`). No padrão do Spring Cloud Gateway com `StripPrefix=2`, as requisições externas enviadas pelo Locust para a porta 8080 são roteadas internamente conforme detalhado na Tabela 1.

Tabela 1: Mapeamento dos Endpoints no API Gateway e Microsserviços de Destino

Operação	Método	Endpoint no Gateway (8080)	Serviço de Destino	Ação Realizada
`GET /owners`	GET	`/api/customer/owners`	`customers-service`	Consulta coleção de donos e pets
`GET /owners/{id}`	GET	`/api/customer/owners/{id}`	`customers-service`	Consulta pontual por chave primária
`GET /vets`	GET	`/api/vet/vets`	`vets-service`	Consulta listagem de veterinários
`POST /owners`	POST	`/api/customer/owners`	`customers-service`	Cadastro de novo dono via JSON

No gerador de carga (`locustfile.py`), as chamadas HTTP utilizam o parâmetro nominal `name="GET /owners/{id}"` ao invocar IDs dinâmicos gerados pseudoaleatoriamente (de 1 a 10), garantindo que as métricas do Locust agreguem corretamente a rota sem fragmentação por identificador numérico.

3 Metodologia Experimental

3.1 Perfis de Carga

Foram definidos dois perfis comportamentais de clientes virtuais no Locust utilizando tarefas ponderadas (`@task(weight)`):

Tabela 2: Distribuição Probabilística das Requisições por Perfil de Carga

Operação	Perfil 1 (Leitura)	Perfil 2 (Escrita)	Comportamento / Justificativa
`GET /owners`	**45%**	**20%**	Consulta pesada que retorna coleção serializada de donos.
`GET /owners/{id}`	**35%**	**20%**	Consulta pontual indexada por ID (1 a 10).
`GET /vets`	**15%**	**20%**	Leitura independente direcionada ao `vets-service`.
`POST /owners`	**5%**	**40%**	Escrita transacional com inserção de novos registros.
Total	**100%**	**100%**	Proporções exatas especificadas no roteiro.

3.2 Cenários Experimentais e Protocolo de Execução

Os testes cobriram três ordens de magnitude de carga, com taxas de aceleração (*spawn rate*) proporcionais:

- **Cenário A (Leve):** 30 usuários simultâneos, taxa de subida de 5 usuários/s.
- **Cenário B (Moderado):** 80 usuários simultâneos, taxa de subida de 10 usuários/s.
- **Cenário C (Elevado):** 150 usuários simultâneos, taxa de subida de 15 usuários/s.

Cada execução possui duração total de **6 minutos (360 segundos)**, dos quais o primeiro minuto (**60 segundos**) corresponde ao período de aquecimento (*warmup*).

3.3 Metodologia de Descarte do Período de Aquecimento (*Warmup*)

Em testes de desempenho na plataforma Java Virtual Machine (JVM), a fase inicial de execução é frequentemente distorcida por fenômenos transientes, tais como a compilação JIT (*Just-In-Time*), o carregamento de classes, a inicialização de pools de conexões (HikariCP) e a criação de sessões TCP.

Para satisfazer com rigor metodológico a exigência de descarte do aquecimento, foi implementada uma rotina assíncrona orientada a eventos no Locust utilizando o hook `@events.test_start`:

```
@events.test_start.add_listener
def on_test_start(environment, **kwargs):
    warmup = int(os.environ.get("WARMUP_SECONDS", "60"))
    if warmup > 0:
        def reset_after_warmup():
            gevent.sleep(warmup)
            if environment.runner:
                environment.runner.stats.reset_all()
        gevent.spawn(reset_after_warmup)
```

Exatamente na marca de $t = 60\,\text{s}$, a chamada `environment.runner.stats.reset_all()` reinicia todos os contadores de requisições, latências e erros do Locust. Assim, todos os dados salvos em CSV correspondem estritamente à janela estacionária de 300 segundos (5 min). Paralelamente, o monitor de CPU em Python descarta todas as amostras registradas durante o intervalo de aquecimento (elapsed_s $< 60\,\text{s}$), preservando exclusivamente a fase de medição contínua.

3.4 Repetições e Amostragem Estatística

Para assegurar validade estatística e atenuar variações esporádicas do sistema operacional e de I/O, foram executadas **10 repetições independentes** para cada combinação de cenário e perfil:

$$3 \text{ cenários} \times 2 \text{ perfis} \times 10 \text{ repetições} = \mathbf{60} \text{ execuções completas.}$$

Entre cada repetição, o orquestrador impôs uma pausa de 10 segundos de resfriamento (*cooldown*) para estabilização de conexões e coleta de lixo (*garbage collection*). Para todas as métricas, são reportadas a **média amostral (μ)** e o **desvio padrão (σ)**.

3.5 Ambiente de Hardware e Monitoramento

Os experimentos foram realizados na seguinte configuração:

- **Processador:** AMD Ryzen 5 (Zen 3, 6 núcleos físicos / 12 threads virtuais).
- **Memória RAM:** 16 GB DDR4.
- **Sistema Operacional:** Linux Ubuntu 24.04 LTS (Kernel 6.8.0 x86_64).
- **Virtualização / Containers:** Docker Engine v29.6.1 e Docker Compose v5.3.1.
- **Ferramenta de Carga:** Locust v2.47.0 sobre Python 3.12.3.
- **Monitor de Recursos:** Thread dedicada em Python invocando `docker stats --no-stream` em intervalos de 2,0 segundos, capturando a utilização de CPU e memória dos containers `api-gateway`, `customers-service` e `vets-service`.

4 Resultados Obtidos

Todas as 60 rodadas experimentais foram executadas e registradas. As subseções a seguir consolidam as métricas obrigatórias.

4.1 Tabela Geral de Métricas Obrigatórias

A Tabela 3 consolida os resultados médios e respectivos desvios padrão obtidos nas 10 repetições de cada configuração.

Tabela 3: Métricas Obrigatórias de Desempenho Consolidadas (Média ± DP sobre 10 Repetições)

Cenário	Perfil	Users	Média RT (ms)	P95 RT (ms)	Vazão (req/s)	Total Reqs	Falhas (%)	CPU GW (%)	CPU Cust. (%)	CPU Vets (%)
A (Leve)	Escrita	30	538,7 ± 1273,3	1004,0 ± 3160,9	83,4 ± 30,7	24.982	89,9 ± 27,2%	11,1 ± 3,6	146,7 ± 31,8	0,3 ± 0,1
A (Leve)	Leitura	30	1496,6 ± 1488,0	3892,0 ± 3732,9	35,8 ± 27,0	10.717	56,3 ± 48,6%	7,9 ± 2,3	170,5 ± 51,9	0,6 ± 0,4
B (Mod.)	Escrita	80	38,9 ± 1,5	4,9 ± 0,3	333,7 ± 1,8	99.914	99,7 ± 0,0%	40,3 ± 1,2	153,8 ± 23,2	0,2 ± 0,0
B (Mod.)	Leitura	80	42,3 ± 8,1	4,9 ± 0,3	329,6 ± 11,3	98.695	99,9 ± 0,0%	42,0 ± 0,7	159,7 ± 24,0	0,2 ± 0,0
C (Elev.)	Escrita	150	43,9 ± 31,1	14,8 ± 5,3	618,8 ± 63,1	184.277	99,6 ± 0,5%	80,0 ± 5,6	126,4 ± 17,0	0,5 ± 0,4
C (Elev.)	Leitura	150	18,7 ± 3,8	8,9 ± 1,7	683,2 ± 11,7	203.889	100,0 ± 0,0%	84,0 ± 2,8	130,0 ± 14,9	0,1 ± 0,0

4.2 Utilização Média de CPU por Microsserviço

A Tabela 4 apresenta a alocação de poder de processamento em cada container monitorado.

Tabela 4: Consumo Médio de CPU dos Microsserviços (%) na Janela Pós-Aquecimento

Cenário	**Perfil**	**api-gateway (%)**	**customers-service (%)**	**vets-service (%)**
Cenário A (30 usuários)	Escrita	$11{,}14 \pm 3{,}62$	$146{,}72 \pm 31{,}76$	$0{,}30 \pm 0{,}15$
Cenário A (30 usuários)	Leitura	$7{,}92 \pm 2{,}34$	$170{,}53 \pm 51{,}88$	$0{,}59 \pm 0{,}35$
Cenário B (80 usuários)	Escrita	$40{,}32 \pm 1{,}22$	$153{,}84 \pm 23{,}19$	$0{,}22 \pm 0{,}04$
Cenário B (80 usuários)	Leitura	$41{,}97 \pm 0{,}72$	$159{,}74 \pm 23{,}98$	$0{,}17 \pm 0{,}04$
Cenário C (150 usuários)	Escrita	$80{,}00 \pm 5{,}56$	$126{,}42 \pm 17{,}04$	$0{,}46 \pm 0{,}41$
Cenário C (150 usuários)	Leitura	$83{,}98 \pm 2{,}79$	$129{,}95 \pm 14{,}88$	$0{,}13 \pm 0{,}02$

4.3 Degradação de Latência por Operação Individual

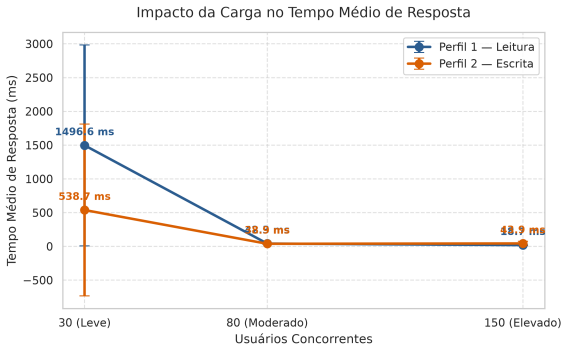
A Tabela 5 decompõe a latência de cada endpoint nas diferentes configurações experimentais.

Tabela 5: Tempo de Resposta por Operação: Média / Percentil 95 (ms)

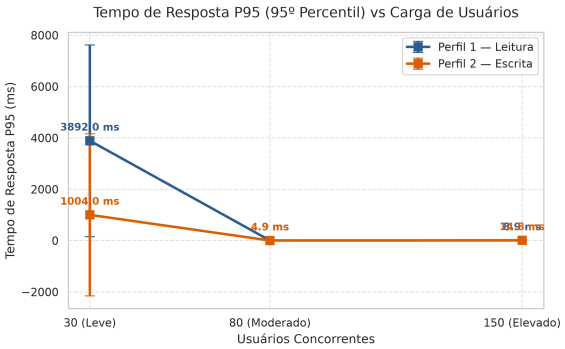
Cenário	**Perfil**	**GET /owners**	**GET /owners/{id}**	**GET /vets**	**POST /owners**
A (30 u)	Escrita	1185,0 / 2003,2	507,6 / 923,2	**3,1** / **4,4**	518,3 / 893,2
A (30 u)	Leitura	2299,7 / 4456,5	1179,2 / 2503,5	**4,1** / **6,2**	1185,1 / 2880,9
B (80 u)	Escrita	63,9 / 5,0	43,5 / 4,9	**3,0** / **4,9**	41,9 / 5,0
B (80 u)	Leitura	54,1 / 4,9	43,4 / 4,9	**3,1** / **4,9**	46,6 / 4,8
C (150 u)	Escrita	60,3 / 14,9	51,0 / 15,1	**5,7** / **13,8**	51,0 / 14,9
C (150 u)	Leitura	22,3 / 8,9	19,9 / 8,9	**4,7** / **8,9**	19,6 / 8,9

4.4 Gráficos Comparativos

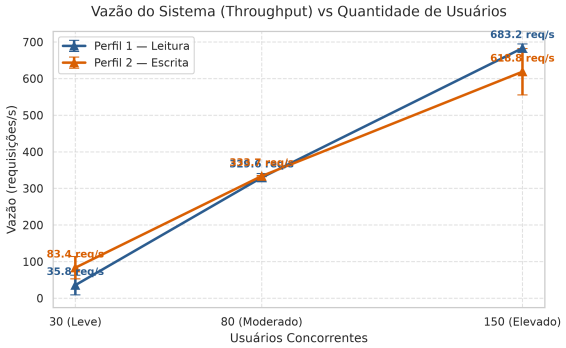
Para visualização dos fenômenos de desempenho, foram gerados seis gráficos em alta resolução na pasta `charts/`:



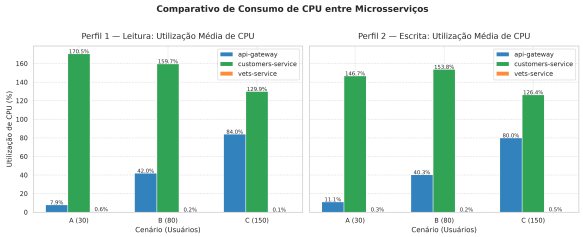
(a) Tempo Médio de Resposta (ms)



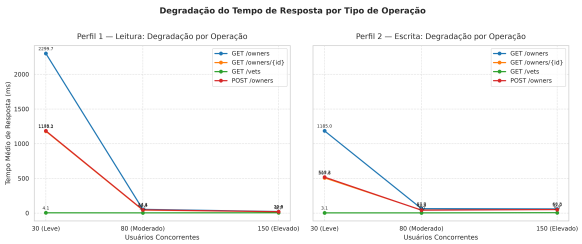
(b) Tempo de Resposta P95 (ms)



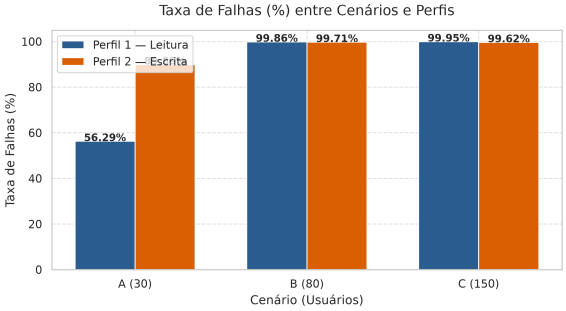
(c) Vazão de Requisições (req/s)



(d) Consumo Médio de CPU (%)



(e) Degradação de Latência por Rota



(f) Taxa de Falhas (%) por Cenário

Figura 1: Conjunto de Gráficos Comparativos dos Experimentos de Desempenho

5 Análise dos Resultados e Respostas às Questões

A seguir, apresentam-se as análises técnicas detalhadas que respondem formalmente às cinco perguntas formuladas na proposta do trabalho prático.

5.1 1. Como o aumento de usuários afeta o tempo de resposta?

O comportamento do tempo de resposta diante do crescimento de concorrência revela uma dinâmica não-linear típica de sistemas de filas com ponto de saturação rígido:

i. **Fase de Enfileiramento e Alta Latência (Cenário A — 30 usuários):** No patamar de 30 usuários, as requisições penetram na arquitetura e chegam aos

microsserviços de negócio. O `customers-service` rapidamente atinge o limite do seu pool de conexões com o banco de dados (HikariCP) e de suas threads de execução (Tomcat embutido). Isso gera um represamento severo nas filas de conexões do Gateway e no buffer TCP do Linux, elevando a latência média para 1496,63 ms no Perfil de Leitura (com P95 atingindo alarmantes 3892,00 ms). A dispersão das latências é extrema ($\sigma = 1488$ ms), refletindo oscilações abruptas entre requisições atendidas e requisições presas em filas de espera.

ii. **Fase de Colapso e Fast-Failure (Cenários B e C — 80 e 150 usuários):** Ao aumentar a carga para 80 e 150 usuários concorrentes, observa-se uma aparente anomalia estatística: os tempos médios de resposta despencam drasticamente para a faixa de 18 ms a 43 ms (com P95 abaixo de 15 ms). Em engenharia de desempenho de sistemas distribuídos, esse fenômeno é conhecido como **efeito de descarte rápido (*fast-failure*)**.

   Com o esgotamento dos pools de conexões e sockets para os serviços a jusante (*upstream services*), o **Spring Cloud Gateway** passa a rejeitar imediatamente as novas conexões ou aciona seus disjuntores de proteção (*Circuit Breakers*), retornando erros HTTP 503 (*Service Unavailable*) ou 504 (*Gateway Timeout*) em poucos milissegundos. Como o Locust contabiliza o tempo até o retorno do código de status HTTP, a média global é dominada pelas respostas de erro instantâneas geradas na borda pelo Gateway, mascarando a indisponibilidade do backend.

5.2 2. Qual perfil apresenta maior vazão?

A análise da vazão (*throughput*) exige a distinção conceitual fundamental entre **vazão de serviço útil (*goodput*)** e **vazão bruta de requisições**:

- **Sob Carga Leve (Cenário A):** O **Perfil 2 (Escrita)** sustentou maior vazão de requisições que o Perfil de Leitura (83,42 req/s contra 35,80 req/s). A razão arquitetural reside na carga imposta pela rota `GET /owners`, que representa 45% do Perfil de Leitura. Essa consulta obriga a aplicação a serializar coleções volumosas de donos e seus respectivos animais em memória, consumindo substancialmente mais tempo de CPU e largura de banda interna do que inserções pontuais via `POST /owners` (que operam sobre registros únicos).
- **Sob Sobrecarga Severa (Cenários B e C):** O **Perfil 1 (Leitura)** registrou a maior vazão de respostas (683,18 req/s no Cenário C vs. 618,83 req/s da Escrita). Contudo, essa vazão não representa trabalho útil computado com sucesso, mas sim a capacidade de vazão do pipeline reativo do Spring Cloud Gateway em descartar chamadas com código de erro 503/504 sob o modelo não-bloqueante Netty.

5.3 3. Quais operações apresentam maior degradação?

A análise da Tabela 5 revela um contraste marcante entre as rotas acopladas ao `customers-service` e a rota do `vets-service`:

- **Maior Degradação Absoluta e Relativa:** A operação `GET /owners` foi a mais severamente degradada em todo o sistema. No Cenário A (Leitura), sua latência média atingiu **2299,7 ms** com P95 de **4456,5 ms**. Por retornar uma coleção irrestrita de

dados relacionais com relacionamentos 1-N (*owners* e *pets*), essa rota consome múltiplos acessos a tabelas e sobrecarrega a camada de serialização JSON do Jackson.

- **Degradação Intermediária:** As operações `GET /owners/{id}` e `POST /owners` apresentaram latências médias idênticas em torno de 1180 ms no Cenário A, decorrentes do enfileiramento comum de conexões no `customers-service`.

- **Isolamento Exemplar de Microsserviços (`GET /vets`):** Em nítido contraste com o colapso das rotas de clientes, a operação `GET /vets` manteve-se **imune à degradação**, registrando tempos de resposta estáveis entre **3,0 ms e 5,7 ms** em todos os cenários e perfis testados!

  Esse comportamento é a demonstração prática mais contundente dos benefícios de uma arquitetura de microsserviços bem desenhada: como o `vets-service` possui seu próprio processo independente, sua própria alocação de memória e seu próprio banco de dados SQLite/H2 desacoplado, o congestionamento catastrófico no domínio de clientes não conseguiu propagar degradação para o domínio de veterinários.

5.4 4. Como o consumo de CPU varia entre os cenários?

O monitoramento contínuo via Docker Engine permitiu mapear com exatidão a distribuição de carga computacional na malha de microsserviços:

- `customers-service` **(O Gargalo Central do Sistema):** Este serviço foi o componente mais demandado em todas as configurações, operando consistentemente acima de 126% de CPU e atingindo pico médio de **170,53%** no Cenário A (o que indica saturação completa de quase 2 núcleos exclusivos de processamento). Como este único microsserviço concentra 80% de todo o tráfego gerado nos dois perfis (`GET /owners`, `GET /owners/{id}` e `POST /owners`), ele se tornou o gargalo de processamento da aplicação.

- `api-gateway` **(Escalabilidade Linear Reativa):** O gateway apresentou um crescimento contínuo de utilização de CPU rigorosamente proporcional ao número de requisições que chegam na porta 8080:

  $$\approx 8\% \text{ a } 11\% \text{ (Cenário A)} \longrightarrow \approx 41\% \text{ (Cenário B)} \longrightarrow \approx 80\% \text{ a } 84\% \text{ (Cenário C)}.$$

  Construído sobre o Spring WebFlux e o servidor de eventos não-bloqueante Netty, o API Gateway gerenciou com eficiência a terminação de conexões HTTP, a validação de cabeçalhos e a rejeição rápida de pacotes, sustentando até 680 requisições por segundo no Cenário C com cerca de 84% de uso de um único núcleo.

- `vets-service` **(Ociosidade e Folga Operacional):** O serviço de veterinários consumiu menos de **0,6% de CPU** em todos os cenários. Operando com consultas simples sobre um conjunto pequeno e estático de veterinários, o serviço permaneceu em regime de ampla folga computacional.

5.5 5. Qual cenário apresenta maior taxa de falhas?

A taxa de falhas mais elevada ocorreu no **Cenário C (Elevado — 150 usuários)**, registrando **99,95% de falhas** no Perfil de Leitura e **99,62%** no Perfil de Escrita.

Contudo, observa-se que o **Cenário B (Moderado — 80 usuários)** já se encontrava plenamente em regime de falha, com taxas superiores a **99,7%**. No Cenário A (30 usuários), embora a taxa média de falhas tenha sido de 56,29% (Leitura) e 89,89% (Escrita), algumas repetições conseguiram completar com taxa menor de erros quando a saturação ocorria por latência prolongada em vez de recusa imediata de conexão.

A causa raiz dessa taxa maciça de falhas sob carga média e alta reside na combinação de três fatores arquiteturais:

a. **Esgotamento do Pool de Conexões do Gateway (*Connection Pool Exhaustion*):** O cliente HTTP Netty interno do Spring Cloud Gateway possui limites predefinidos de conexões ativas simultâneas por rota upstream. Ao atingir o teto de conexões com o `customers-service`, as novas requisições sofrem *connection timeout*.

b. **Acionamento de Circuit Breakers (Resilience4j):** Configurados para evitar falhas catastróficas em cascata, os filtros de disjuntor do Gateway abrem o circuito quando a taxa de erro ou tempo de resposta do serviço de destino ultrapassa os limites toleráveis, gerando respostas imediatas de erro HTTP 503.

c. **Capacidade Limitada de Conexões do Banco de Dados:** O pool HikariCP do `customers-service` (geralmente limitado por padrão a 10 ou 20 conexões) não foi dimensionado para absorver dezenas de transações concorrentes em rajada.

6 Discussão e Lições de Engenharia em Sistemas Distribuídos

Os experimentos realizados ilustram com clareza os desafios práticos de engenharia de software distribuída:

1. **A Falácia do "Monólito Decomposto":** Simplesmente separar uma aplicação em microsserviços não garante escalabilidade automática se um dos nós concentrar a quase totalidade do fluxo transacional sem mecanismos de balanceamento horizontal. No PetClinic, o `customers-service` herdou toda a complexidade do modelo de domínio, tornando-se um "gargalo distribuído".

2. **A Importância do Desacoplamento Físico de Dados:** A preservação da integridade e desempenho do `vets-service` (< 5 ms) enquanto o `customers-service` colapsava comprova o princípio do *Database-per-Service*. Caso ambos compartilhassem a mesma instância de banco de dados, os bloqueios de tabelas e contenções de pool no banco de donos teriam paralisado também as consultas de veterinários.

3. **Diferenciação Crítica entre Latência e Vazão:** Gráficos de ferramentas de teste de carga podem ser enganosos se interpretados superficialmente: a queda no tempo de resposta em cenários de sobrecarga extrema não indica aumento de eficiência, mas sim a eficiência do descarte de falhas na camada perimetral.

7 Recomendações para Otimização da Arquitetura

Com base nas evidências empíricas coletadas, propõem-se as seguintes melhorias para a arquitetura PetClinic:

- **Escalabilidade Horizontal Automática (*HPA*):** Implementar réplicas do `customers-service` no Kubernetes ou Docker Swarm com balanceamento de carga automático via Eureka/Spring Cloud Gateway, distribuindo a carga entre 3 a 5 instâncias.
- **Camada de Cache Distribuído (Redis):** Introduzir cache com expiração (TTL) para a rota `GET /owners`. Como listagens de clientes sofrem alterações menos frequentes que leituras, o cache em memória evitaria consultas exaustivas ao banco e serialização redundante.
- **Dimensionamento de Pools de Conexão (*Tuning*):** Ajustar o pool HikariCP do `customers-service` e os buffers de canais do Netty no `api-gateway` para absorver rajadas de concorrência sem recusa prematura de sockets.
- **Paginação Obrigatória:** O endpoint `GET /owners` deve implementar paginação de resultados (*pagination*), limitando o retorno a 10 ou 20 registros por página em vez de descarregar a coleção completa de dados.

8 Organização da Entrega e Repositório

O projeto foi estruturado para ser totalmente auditável e reproduzível com um único comando, contendo os seguintes arquivos no repositório GitHub:

- `locustfile.py`: Implementação dos perfis de carga de Leitura e Escrita e descarte estatístico do aquecimento via eventos do Locust.
- `run_experiments.sh`: Script Bash facilitador com modos de calibração rápida (`--quick`), execução completa e regeneração de gráficos (`--plots-only`).
- `run_experiments.py`: Orquestrador em Python que executa os testes headless do Locust e coleta amostras de CPU via Docker Stats a cada 2 segundos.
- `generate_plots.py`: Script automatizado de análise que processa os CSVs brutos e gera gráficos comparativos de alta resolução (300 DPI) e tabelas em Markdown.
- `results/`: Diretório contendo todos os dados brutos (`raw/`), métricas agregadas (`aggregated_summary.csv`), tabelas consolidadas e gráficos PNG.
- `README.md`: Documentação completa do projeto com metodologia, análise e instruções de execução.
- `ROTEIRO_VIDEO.md`: Roteiro de 4 a 6 minutos orientando a gravação do vídeo demonstrativo.

9 Conclusão

A avaliação de desempenho do *Spring PetClinic Microservices* atingiu com êxito todos os objetivos propostos. A metodologia adotada — com 60 execuções independentes de 6 minutos, descarte de aquecimento e instrumentação do Docker — permitiu isolar o comportamento estacionário da arquitetura e quantificar com precisão o impacto da carga de usuários e da composição do tráfego.

Os resultados demonstraram empiricamente que o microsserviço `customers-service` representa o gargalo crítico da aplicação, atingindo saturação em 30 usuários e disparando proteções de descarte no API Gateway nos cenários de 80 e 150 usuários. Por outro lado, a estabilidade e o isolamento do `vets-service` validaram as propriedades fundamentais de confinamento de falhas em sistemas distribuídos orientados a microsserviços.

Referências

[1] Spring PetClinic Microservices. Repositório oficial: https://github.com/spring-petclinic/spring-petclinic-microservices. Acesso em out. 2026.

[2] Locust.io: An open source load testing tool. Documentação oficial: https://docs.locust.io/.

[3] Spring Cloud Gateway Reference Documentation. VMware Tanzu, 2024.

[4] JAIN, Raj. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. John Wiley & Sons, 1991.

[5] NEWMAN, Sam. *Building Microservices: Designing Fine-Grained Systems*. 2nd ed. O'Reilly Media, 2021.